

SWE 580 Term Project Report

Tool Granularity in LLM-Based Knowledge Retrieval

Sena Oz

Implementation: Direct API function calling (Option 2)

The starter code provides a 100-note markdown vault with a Whoosh search backend (10-result cap per query) and an evaluator. The task is to design two tool interfaces on top of this backend, evaluate them on 25 ground-truth queries, and compare which design helps the LLM succeed — and why.

- **Config A** — 4 coarse-grained tools, combinable parameters
- **Config B** — 9 fine-grained tools, one operation each
- **Model:** google/gemini-2.5-flash via OpenRouter API

The Two Configurations

Config A — 4 Tools

Tool	What it does
search_notes	Full-text + tags + date range in one call — all parameters optional and combinable
get_note	Retrieve by title or path
get_related_notes	Incoming/outgoing/both links via direction param
get_vault_overview	Vault stats + recent notes

search_notes applies all filters at the backend before the 10-result cap. A query combining tags and dates hits the backend once; the cap applies to the already-filtered result set.

Config B — 9 Tools

Tool	What it does
search_by_content	Full-text search
search_by_tags	Filter by tags (AND logic)
search_by_date	Filter by date range

get_note_by_title	Retrieve by title
get_note_by_path	Retrieve by path
get_incoming_links	Backlinks to a note
get_outgoing_links	Links from a note
get_vault_stats	Vault statistics
get_recent_notes	Most recent notes

Each tool does one thing. Combined queries require multiple calls and LLM-side intersection of results.

The Cap Intersection Problem

With a 10-result cap, Config B's intersection pattern is structurally vulnerable:

1. search_by_tags(["deep-learning"]) → top 10 deep-learning notes
2. search_by_content("neural networks") → top 10 content matches
3. Intersection in the LLM's context window

If the target note ranks 11th in either list, it disappears from both and cannot be recovered. Config A avoids this — all filters go to the backend together before the cap is applied.

Evaluation Setup

- **25 test queries** with ground-truth expected paths
- **Metric:** Exact match — extracted paths must equal expected paths exactly
- **Secondary metrics:** tool calls per query, total tokens, latency

Category	Count
simple_lookup	2
tag_search	2
temporal	3
content_search	2

multi_faceted	9
graph_based	7

Prompt Engineering

Both configs started with the same 2-line baseline prompt. Failures were diagnosed from trace logs — round-by-round tool call history stored in each result JSON.

Config A Iterations

v0 → 40% (10/25)

Root causes from trace analysis:

- LLM found correct notes via tool calls but final response contained no .md paths — evaluator extracted nothing
- Treated folder names as tag values (search_notes(tags: ["research"]) → empty)
- Title format unknown — guessed "Reinforcement Learning Research" instead of "Reinforcement Learning"
- No tool selection guidance for graph queries

v1 → 88% (22/25) +48pp

Added: output format (RESULT_PATHS: [...]), vault structure, title convention, folder/tag distinction, tool usage guide.

The output format instruction alone fixed ~12 queries.

v2 → 84% (21/25) -4pp — regression

Added folder-restricted query pattern. New instruction "do not add folder names as query text" caused q23 to fail — LLM stopped using search_notes(tags: ["python"], query: "folder:reference") and returned empty instead.

v3 → 84% (21/25) — still failing q23

Added reference and meeting to tag list. LLM still did not connect "reference tag = reference/ folder" without an explicit example.

v4 → 88% (22/25) — best Config A

Clarified that reference and meeting are BOTH folder names AND valid tags, with explicit example: search_notes(tags: ["python", "reference"]). This fixed q23.

Version	Success	Exact	Avg Tokens	Key change
v0	40%	10/25	2,556	Baseline
v1	88%	22/25	3,420	Output format + structure

v2	84%	21/25	3,973	Folder pattern — regression
v3	84%	21/25	3,939	reference tag — still failing
v4	88%	22/25	3,981	reference/meeting tag exception

Config B Iterations

v0 → 52% (13/25)

Same output format issue as Config A, but better baseline overall — explicit tool names (search_by_date, get_incoming_links) were self-documenting. The LLM selected correct tools for several queries without any guidance.

v1 → 84% (21/25) +32pp

Added: output format, vault structure, tool selection table, title convention.

v2 → 88% (22/25) +4pp

Added: folder-restricted query pattern, outgoing-links filter to research/ for prerequisite queries, graph+folder incoming link pattern.

v3 → 92% (23/25) +4pp — best Config B

Added tag+content pattern: use search_by_content first, then filter by tag from results — instead of intersecting two capped searches. This fixed q15 where the dual-cap intersection was excluding notes ranked beyond position 10 in one of the lists.

v4 → 88% (22/25) -4pp — regression

Over-specified q18 and q22 patterns. New instructions conflicted with q12, which had been passing. Stayed with v3.

Version	Success	Exact	Avg Tokens	Key change
v0	52%	13/25	3,466	Baseline
v1	84%	21/25	4,051	Output format + tool table
v2	88%	22/25	4,431	Folder + link patterns
v3	92%	23/25	4,294	Content-first tag+content
v4	88%	22/25	4,627	Over-specified — regression

Results

Head-to-Head (Best Versions)

Config A v4 vs Config B v3, Gemini 2.5 Flash:

Metric	Config A	Config B	Winner
Success rate	88% (22/25)	92% (23/25)	B
Avg precision	0.987	1.000	B
Avg recall	0.960	0.960	Tie
Avg F1	0.964	0.972	B
Avg tool calls	1.08	1.12	A
Avg tokens	3,981	4,294	A
Avg latency	1.668s	1.674s	Tie

By Category

Category	Config A	Config B	Winner
simple_lookup (2)	100%	100%	Tie
tag_search (2)	100%	100%	Tie
temporal (3)	100%	100%	Tie
content_search (2)	100%	100%	Tie
multi_faceted (9)	89%	89%	Tie
graph_based (7)	71%	86%	B

Config B wins on graph queries (6/7 vs 5/7). Everything else is tied.

Per-Query Breakdown

Query	Category	A	B	Tools A	Tools B	Tokens A	Tokens B
q01	simple_lookup	✓	✓	1	1	3,657	3,959
q02	simple_lookup	✓	✓	1	1	3,606	3,908
q03	tag_search	✓	✓	1	1	3,579	3,875
q04	tag_search	✓	✓	1	1	4,247	4,543
q05	temporal	✓	✓	1	1	3,574	3,870
q06	temporal	✓	✓	1	1	3,807	4,103
q07	content_search	✓	✓	1	1	3,584	3,880
q08	content_search	✓	✓	1	1	3,395	3,691
q09	multi_faceted	✓	✓	1	1	4,294	4,525
q10	graph_based	✓	✓	1	1	4,259	4,539
q11	multi_faceted	✓	✓	1	1	3,725	4,450
q12	multi_faceted	✓	✓	1	1	3,514	3,878
q13	multi_faceted	✓	✓	1	1	4,303	4,599
q14	multi_faceted	✓	✓	1	1	4,144	4,374
q15	multi_faceted	✓	✓	1	1	3,901	4,271
q16	graph_based	✗	✓	1	1	3,670	3,947
q17	graph_based	✓	✓	1	2	4,058	4,387
q18	graph_based	✗	✗	1	1	4,031	4,311
q19	graph_based	✓	✓	1	1	3,835	4,015

q20	graph_based	✓	✓	1	1	4,273	4,553
q21	graph_based	✓	✓	3	3	6,065	6,500
q22	multi_faceted	✗	✗	1	1	4,215	4,511
q23	multi_faceted	✓	✓	1	1	3,887	4,183
q24	multi_faceted	✓	✓	1	1	3,791	4,087
q25	temporal	✓	✓	1	1	4,100	4,396

Where configs diverge:

- **q16** (outgoing links of Transformers for prerequisites): A ✗, B ✓ — Config A's `get_related_notes(direction: "outgoing")` returned `Paper_Recommender` as a false positive (Transformers.md links to it but it's a project note). Config B's `get_outgoing_links` + explicit `research-filter` instruction excluded it correctly.
- **q18** (project notes related to attention research): Both fail — `Paper_Recommender` and `Text_Summarizer` link to Transformers but not directly to `Attention_Mechanisms`. Neither config's multi-hop traversal reaches them reliably.
- **q22** (meeting notes discussing BERT): Both fail — 8 meeting notes mention BERT, search results cap at 10 total (across all folders), `Advisor_Meeting_Feb07` consistently ranks outside the top positions.

Config A is 313 tokens cheaper per query on average (3,981 vs 4,294, -7%). The gap is small because at best versions, both configs average ~1.1 tool calls per query — there's little intersection overhead left.

Findings

1. Output format is the single most impactful prompt element

At baseline, both configs were making correct tool calls but reporting answers in prose. The evaluator's path extractor found nothing. Adding `RESULT_PATHS: [...]` fixed ~12 queries for Config A (40% → 88%) in one step. No other single change came close.

This is non-obvious: the tool interface was working correctly. The failure was in the output contract — how results are reported back to the caller.

2. Fine-grained tool names work better at baseline

Config B scored 52% at baseline vs Config A's 40%, despite having more tools (more decision points). `search_by_date`, `get_incoming_links`, `get_outgoing_links` — each name unambiguously describes one

operation. The LLM selected them correctly for temporal and graph queries before any guidance was given.

Config A required the LLM to know that `search_notes` accepts `date_from/date_to` parameters, and that `get_related_notes` takes a direction enum — neither discoverable from the tool name.

3. Config B reached higher peak accuracy (92% vs 88%)

The best Config B prompt outperformed the best Config A prompt. The critical fix in Config B v3 — using `search_by_content` first for tag+content queries, then filtering by tag from results rather than intersecting two capped searches — produced a result that Config A's single combined `search_notes` call could not replicate in the same way.

Config A's `search_notes` applies all filters at the backend before the cap. For tag+content queries this should be equivalent. But q15 passed for both best versions, so the real difference was q16: Config B's explicit research/-filter instruction for outgoing links cleanly excluded the false-positive project note. Config A's coarser `get_related_notes` tool doesn't expose this distinction easily.

4. The cap is a structural problem for Config B's intersection pattern, but it's addressable by prompt

The dual-cap intersection failure (two capped searches, target notes in neither top-10) can be fixed without changing the tool design. The content-first approach — run `search_by_content`, then filter results by tag — performs the filtering in a single capped search rather than intersecting two. This converts a structural disadvantage into a prompt engineering problem.

The fix requires knowing that tag fields are included in search results (so filtering is possible after the fact). This is not obvious from the tool description alone — it's operational knowledge about the data format.

5. Over-specification reliably causes regression

Both configs showed this pattern: adding detailed instructions to fix one failure type broke queries that were already passing. Config A v2 broke q23 by forbidding folder names in queries. Config B v4 broke q12 with conflicting intersection instructions.

The working pattern: diagnose one failure type per iteration, add the minimal instruction that addresses it, verify nothing regresses. Adding more than one new rule per iteration makes it hard to isolate regressions.

6. Some queries are structurally unfixable with keyword search

q18 (project notes related to attention research): `Paper_Recommender` and `Text_Summarizer` are project notes that use transformer-based models. They link to `Transformers` but not `Attention_Mechanisms`. "Related to attention research" is a semantic judgment — not expressible as a tag filter, content keyword, or direct link traversal. This query cannot be reliably solved by either config without embedding-based semantic search.

q22 (meeting notes discussing BERT): 8 meeting notes mention BERT. 3 are "expected." The distinction is qualitative (BERT is the main topic vs mentioned in passing), but with identical keyword matches, there is no ranking signal that separates them consistently. Both configs failed this query in every version.

Limitations

- 25 queries is small; categories with 2 queries have no statistical power
- Prompts were tuned by observing failures — the best-version numbers overstate zero-shot performance. Baseline numbers (40%/52%) are more representative of what an engineer would see on first deployment
- Only one model tested (Gemini 2.5 Flash). Other models may behave differently on tool selection and output formatting
- The 10-result cap is hardcoded. Higher caps would change the structural disadvantage profile

Summary

Finding	Verdict
Output format	Most impactful single change — fixes output contract failures before any tool design matters
Baseline usability	Config B wins — explicit tool names require less prompt guidance
Peak accuracy	Config B wins — 92% vs 88% after full tuning
Token efficiency	Config A wins — 313 tokens cheaper per query at best versions
Structural cap issue	Addressable by prompt (content-first approach), not just a design problem
Unfixable queries	q18 (semantic relatedness), q22 (qualitative content distinction) — neither config solves these

Config B v3 is the best-performing configuration. Config A v4 uses fewer tokens but loses on graph queries. For production use where token cost matters and queries are mostly simple or single-dimension, Config A is adequate. Where accuracy on graph-based and multi-faceted queries is the priority, Config B with proper prompt engineering is the better choice.

Config A best prompt: config_a_prompt.txt (v4). Config B best prompt: config_b_prompt.txt (v3). All run results and prompt snapshots in EXPERIMENT_LOG.md. Raw JSON in results/.